

Problem Set #5

Due date: October 9, 2026 11:59 PM

REMINDER: As you work on your coding assignments, it is important to remember that passing the examples provided does not guarantee full credit. While these examples can serve as a helpful starting point, it is ultimately your responsibility to create additional tests, ensure that it is functioning correctly and meets all the requirements and specifications outlined in the assignment instructions. Your grade for this assignment is divided in two parts:

50% > Passing all post-due date tests
(no partial credit for failed cases)

50% > Clarity and design of your code
(Class Coding Standards, requirements/specifications)

Before you start...

- Your code must be written using Python language.
- Function names must be defined as stated in the function description. Failure to comply with this requirement will result in a 0 score for the visible and post-due date testing.
- No code should be written outside functions, including function calls. All testing code must be done inside the `main` function
- Your functions should NOT have any `input()` calls in its body.
- Your code should be concise and efficient without useless code or redundant code.
- Failure to follow the assignment's requirements will result in a zero score for the assignment even if the autograder gives you full score.
- Code submitted with syntax errors does not receive ANY credit.
- Revise the Grading Notes provided in the Canvas Assignment that includes the evaluation criteria for Clarity and Design.
- You will be writing several functions, but they will all be saved in one file: `PS5.py`. Please save all the functions in this one file or you will lose points.
- Ask questions using our Problem Set Questions channel in Microsoft Teams.

REMINDER: For all the problems in this assignment, you are not allowed to use material that has not been covered in class.

- The only string methods you are allowed to use are: `split`, `strip` and `join`
 - The only list methods you are allowed to use are: `append`, `pop` and `del`
 - You can use the `len` function and slicing syntax only in the form covered in lectures
 - Think about how you can organize your solution into logical steps by breaking it down into smaller, focused tasks (helper functions), **it is part of your grade!**
 - You are only allowed to use the following data types: `int`, `float`, `bool`, `str`, `list` and `tuple`.
- ➡ To receive any credit for any of the following problem, you are not allowed to use any built-in functions, modules, or list/string methods that we have not cover in class.

You are a developer building a lightweight personal data dashboard, a tool that ingests raw text feeds, processes environmental sensor logs, and manages user profile records. Over the course of this assignment you will build the components of that dashboard piece by piece. Several functions you write early will be called inside later ones, read through all problems before you start coding.

Part 1: Warm-up

Problem 1.1 (10 pts) Your dashboard pulls in short text posts from a social feed API. Unfortunately, the raw feed is messy, posts often arrive with leading and trailing whitespace, and some posts are prefixed with a category tag formatted as "[TAG] " (a word in square brackets followed by a space). Write the function `clean_post` that takes a single string post and returns a cleaned version of it: whitespace stripped from both ends, and if the post begins with [, the tag and the space immediately after it are removed.

Example:

```
>>> clean_post("      [NEWS] Rainfall expected this weekend      ")
'Rainfall expected this weekend'
```

Problem 1.2 (10 pts) Environmental sensors in your dashboard transmit readings as a formatted string:

`"SENSOR_ID:VALUE:UNIT"`

For example: `"T42:23.7:C"` means sensor T42 recorded a value of 23.7 in degrees Celsius. Write the function `parse_sensor` that takes a single string reading and returns a tuple of three elements: sensor id, numerical value and unit.

Example:

```
>>> parse_sensor("H07:58.2:km")
('H07', 58.2, 'km')
```

Problem 1.3 (5 pts) Your dashboard receives a batch of raw sensor reading strings (as a list) and needs to build a structured log as a list of parsed tuples for further processing. Write the function `build_sensor_log` that takes a list of raw reading strings readings and returns a new list where each element is the parsed tuple produced by calling `parse_sensor` on the corresponding raw string.

Example:

```
>>> build_sensor_log(["T42:23.7:C", "H07:58.2:PCT", "P11:1013.25:hPa"])
[('T42', 23.7, 'C'), ('H07', 58.2, 'PCT'), ('P11', 1013.25, 'hPa')]
```

Part 2: Level-Up

Problem 2.1 (10 pts) Your dashboard stores user profiles as tuples in the format (username, email, city). Because profiles come from different input sources, they are often inconsistently formatted. Write the function `normalize_profile` that takes a profile tuple and returns a new tuple with username stripped of whitespace, email converted to all lowercase and stripped of whitespace, and city stripped of whitespace.

Example:

```
>>> normalize_profile(("  janed92  ", "      Jane_Doe@GMAIL.COM", "  State College  "))
('janed92', 'jane_doe@gmail.com', 'State College')
```

Problem 2.2 (10 pts) Given a sensor log (a list of tuples as produced by `build_sensor_log`), your dashboard needs to flag any reading whose value falls outside an acceptable range. Flagging means replacing the tuple at that position with a new tuple where the sensor id has "*" prepended to it. For example, a reading ('T42', 23.7, 'C') whose value is outside the range becomes ('*T42', 23.7, 'C'). Write the function `flag_readings` that takes a sensor log list `log`, a float `low`, and a float `high`, and modifies the list in place without returning anything.

Example:

```
>>> log = [('T42', 23.7, 'C'), ('H07', 110.0, 'PCT'), ('P11', 1013.25, 'hPa')]
>>> flag_readings(log, 0.0, 100.0)
>>> log
[('T42', 23.7, 'C'), (*H07', 110.0, 'PCT'), (*P11', 1013.25, 'hPa')]
```

Problem 2.3 (10 pts) Your dashboard receives a list of raw post strings. You need to produce a cleaned feed (a new list where every post has been cleaned using `clean_post` from Problem 1.1), and any post that is an empty string after cleaning is excluded entirely. Write the function `clean_feed` that takes a list of raw post strings `posts` and returns a new list of cleaned, non-empty posts.

Example:

```
>>> clean_feed(["  [NEWS] Rain expected  ", "  We ARE !!! ", "[UPDATE] New bike lanes added"])
['Rain expected', 'We ARE !!!', 'New bike lanes added']
```

Part 3: Putting It All Together

Problem 3.1 (10 pts) Your dashboard needs to generate a summary report of a cleaned post feed. A summary report is a list of tuples, one per post, where each tuple contains the number of words in the post, the first word and the last word. Write the function `feed_summary` that takes a list of post strings and returns the list of summary tuples.

Example:

```
>>> feed_summary(["  [NEWS] Rain expected today ", "  New bike lanes added"])
[(3, 'Rain', 'today'), (4, 'New', 'added')]
```

Problem 3.2 (10 pts) Before posts are sent to a human moderator for review, your dashboard's moderation tool must obscure sensitive characters to protect user identity. The characters to mask are provided as a tuple, a fixed, read-only collection chosen deliberately because the masking rules should not change during processing. Each post in the feed is stored as a list of words. Write the function `mask_words` that takes a list of strings and a tuple of characters, and mutates the list in place so that every character in the tuple (case-insensitive) found in each word is replaced with "*".

Example:

```
>>> words = ["Rainfall", "expected", "downtown"]
>>> mask_words(words, ('a', 'E', 'o'))
>>> words
['R*inf*ll', '*xp*ct*d', 'd*wnt*wn']
```

Problem 3.3 (15 pts) You have spent this entire assignment building the tools that power a personal data dashboard, a system that watches two live data streams simultaneously: a community text feed and an environmental sensor network. The sensor network is the dashboard's early warning system. When everything is normal, the feed is displayed as-is. But when the sensors detect abnormal readings (values outside the acceptable range), the dashboard automatically switches into secure mode: all posts in the community feed are scrubbed of sensitive characters before being displayed to a moderator. The idea is simple: if something is wrong in the physical environment, you lock down the information layer too.

Your final task now is to write the function `dashboard_report`, the brain of the entire system. It receives a raw post feed, a batch of raw sensor readings, an acceptable sensor range, and a tuple of characters to mask. It runs both pipelines end to end, decides whether secure mode is triggered, and returns a single status string that summarizes the state of the whole dashboard. When the dashboard reports its status, it “speaks” in a single line. That line tells you how many sensors were checked, how many were healthy, how many triggered the alarm, how many posts made it through the feed, what the average post length was (the average word count across summary feeds, rounded to 1 decimal place), and whether the system had to switch into secure mode. For example, a dashboard watching three sensors, one of which flagged an abnormal reading, over a feed of four posts averaging six words each would report:

"SENSORS: 3, OK: 2, FLAGGED: 1 | FEED: 4, AVG_WORDS: 6.0, MASKED: YES"

If all sensors are healthy the report looks the same, but closes with `MASKED: NO`. This function does not introduce any new logic of its own, everything it needs, you have already written. Your job is to connect the pieces in the right order and let the system speak for itself. The format of the output string must match exactly.

Example:

```
>>> dashboard_report(
...     [" [NEWS] Rainfall expected ", " [UPDATE] New bike lanes "], # feed
...     ["T42:23.7:C", "H07:110.0:PCT", "P11:50.0:hPa"],           # sensors
...     0.0, 100.0, ('a', 'E', 'i', 'O', 'u')
... )
'SENSORS: 3, OK: 2, FLAGGED: 1 | FEED: 2, AVG_WORDS: 2.5, MASKED: YES'
```